

03 — Date & Time Data Types in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Overview of Date/Time Types](#)
 3. [date](#)
 4. [time / time with time zone](#)
 5. [timestamp / timestampz](#)
 6. [interval](#)
 7. [Timezone Handling Deep Dive](#)
 8. [Date/Time Arithmetic](#)
 9. [Extraction and Truncation](#)
 10. [Special Values and Constants](#)
 11. [Storage Reference](#)
 12. [SQL Examples](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Performance Considerations](#)
 16. [Interview Questions & Answers](#)
 17. [Exercises with Solutions](#)
 18. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Choose between `timestamp` and `timestampz` correctly and explain the difference
 - Perform date/time arithmetic using intervals
 - Handle timezone conversions accurately
 - Extract parts of dates and timestamps
 - Avoid the most common timezone bugs in applications
 - Index and query date/time columns efficiently
-

Overview of Date/Time Types

Date/Time Type Family

— <code>date</code>	– calendar date only (no time, no timezone)
— <code>time [(p)]</code>	– time of day only (no date, no timezone)
— <code>time [(p)] with time zone</code>	– time of day with timezone offset
— <code>timestamp [(p)]</code>	– date + time (no timezone)
— <code>timestamp [(p)] with time zone (= timestampz)</code>	– date + time + UTC offset
— <code>interval [(p)]</code>	– a duration (difference between two timestamps)

`p` = optional fractional seconds precision (0–6), default is 6 microseconds.

date

Definition

Stores a calendar date: year, month, day. No time component, no timezone.

- **Storage:** 4 bytes
- **Range:** 4713 BC to 5874897 AD
- **Resolution:** 1 day

Internal representation: integer count of days since 2000-01-01

```
2024-01-15 → day 8780
2000-01-01 → day 0
1999-12-31 → day -1
```

Input formats

```
SELECT '2024-01-15'::DATE;      -- ISO 8601 (recommended)
SELECT 'January 15, 2024'::DATE; -- verbose format
SELECT '15/01/2024'::DATE;      -- locale-dependent, avoid
SELECT 'today'::DATE;           -- current date
SELECT 'tomorrow'::DATE;
SELECT 'yesterday'::DATE;
```

time / time with time zone

time [(p)]

- **Storage:** 8 bytes
- **Range:** 00:00:00 to 24:00:00
- **Resolution:** 1 microsecond

```
SELECT '14:30:00'::TIME;
SELECT '14:30:00.123456'::TIME; -- with microseconds
SELECT CURRENT_TIME;             -- current time of day
SELECT LOCALTIME;                -- current local time
```

time with time zone (timetz)

- **Storage:** 12 bytes (8 bytes time + 4 bytes UTC offset)
- **Note:** `timetz` is practically useless — a time-of-day with a timezone offset without a date makes no sense for DST-aware calculations. PostgreSQL retains it for SQL standard compliance only.
- **Recommendation:** Avoid `timetz`. Use `timestamptz` instead.

timestamp / timestamptz

timestamp [(p)] — WITHOUT time zone

- **Storage:** 8 bytes
- **Range:** 4713 BC to 294276 AD
- **Resolution:** 1 microsecond

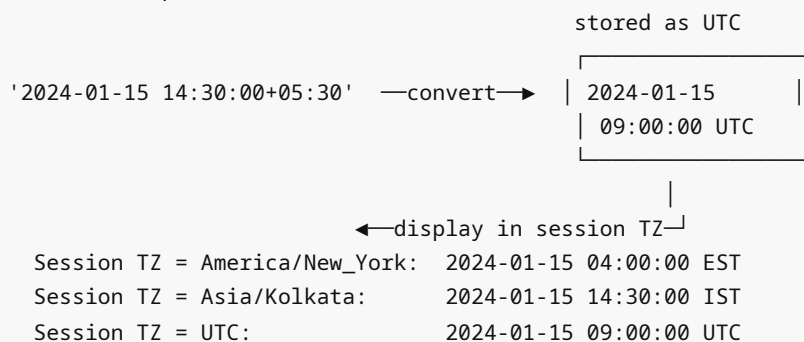
- **Semantics:** Stores a "wall clock" reading with no timezone context. PostgreSQL does not know what timezone this was recorded in.

timestamp internal representation:
 microseconds since 2000-01-01 00:00:00 UTC
 (stored as 8-byte integer)

timestamptz — WITH time zone

- **Storage:** 8 bytes (same as timestamp!)
- **Behavior:** PostgreSQL **converts the input to UTC** at storage time and **converts to the session timezone** at display time.
- **This is the recommended type** for almost all timestamp needs.

How timestamptz works:



interval

Definition

An interval represents a **duration** — a span of time. It is not anchored to any specific point in time.

- **Storage:** 16 bytes
- **Range:** -178,000,000 years to +178,000,000 years
- **Components:** months, days, microseconds (stored separately!)

interval internal structure (16 bytes):

months (4 B)	days (4 B)	microseconds (8 B)
--------------	------------	--------------------

Why store months, days, and microseconds separately? Because:

- 1 month ≠ a fixed number of days (months have 28–31 days)
- 1 day ≠ a fixed number of seconds (DST changes ±1 hour)

Interval input formats

```

SELECT INTERVAL '2 years 3 months 5 days';
SELECT INTERVAL '2 years';
SELECT INTERVAL '3 months';
  
```

```
SELECT INTERVAL '5 days';
SELECT INTERVAL '12 hours 30 minutes';
SELECT INTERVAL '00:30:00';
SELECT INTERVAL 'P2Y3M5DT12H30M0S'; -- ISO 8601 duration format
SELECT INTERVAL '1 week';           -- = 7 days
SELECT '2 years 3 mons 5 days'::INTERVAL;
```

Timezone Handling Deep Dive

How PostgreSQL stores timestamps

```
timestamp (no TZ):
  Stores exactly what you give it.
  No conversion. No timezone awareness.
  Same value regardless of session timezone.

timestampz:
  Converts input to UTC on storage.
  Converts UTC to session timezone on display.
  The stored value is always UTC.
```

Session timezone

```
-- View current session timezone
SHOW timezone;

-- Set session timezone (affects display of timestampz)
SET timezone = 'America/New_York';
SET timezone = 'UTC';
SET timezone = 'Asia/Kolkata';
SET timezone = '+05:30';

-- View all available timezones
SELECT * FROM pg_timezone_names ORDER BY name;
```

AT TIME ZONE operator

```
-- Convert timestampz to specific timezone
SELECT now() AT TIME ZONE 'Asia/Kolkata';
SELECT now() AT TIME ZONE 'America/Los_Angeles';

-- Convert a local timestamp to timestampz (interpret as specific zone)
SELECT '2024-01-15 14:30:00'::TIMESTAMP AT TIME ZONE 'US/Eastern';
-- Returns: 2024-01-15 19:30:00+00

-- Double conversion: local to UTC then to another zone
SELECT ('2024-01-15 14:30:00'::TIMESTAMP AT TIME ZONE 'US/Eastern')
      AT TIME ZONE 'Asia/Tokyo';
```

DST (Daylight Saving Time) example

```
-- What happens during "spring forward" (2 AM → 3 AM)?
SET timezone = 'America/New_York';
SELECT '2024-03-10 02:30:00'::TIMESTAMPTZ;
-- PostgreSQL handles this correctly: no 2:30 AM exists on this date
-- It resolves ambiguous/nonexistent times systematically

-- What happens during "fall back" (2 AM → 1 AM again)?
SELECT '2024-11-03 01:30:00 America/New_York'::TIMESTAMPTZ;
-- Without explicit offset, PostgreSQL picks one (usually standard time)
-- Be explicit: '2024-11-03 01:30:00-04' vs '2024-11-03 01:30:00-05'
```

Date/Time Arithmetic

```
-- Date arithmetic
SELECT '2024-03-15'::DATE + 30; -- adds 30 days
SELECT '2024-03-15'::DATE - '2024-01-01'::DATE; -- difference in days (74)
SELECT '2024-03-15'::DATE + INTERVAL '3 months'; -- date + interval =
timestamp
SELECT '2024-03-15'::DATE - INTERVAL '1 year 2 months'; -- subtract interval

-- Timestamp arithmetic
SELECT now() + INTERVAL '1 hour';
SELECT now() - INTERVAL '7 days';
SELECT '2024-12-31 23:59:59'::TIMESTAMP + INTERVAL '1 second';

-- Interval arithmetic
SELECT INTERVAL '2 hours' + INTERVAL '30 minutes'; -- 2:30:00
SELECT INTERVAL '1 year' - INTERVAL '3 months'; -- 9 mons
SELECT INTERVAL '1 day' * 7; -- 7 days
SELECT INTERVAL '1 week' / 7; -- 1 day

-- Age calculation
SELECT age('2024-01-15', '1990-05-20'); -- "33 years 7 mons 26 days"
SELECT age(CURRENT_DATE, birth_date) FROM people;

-- Difference between two timestamps
SELECT '2024-01-15 14:30:00'::TIMESTAMP - '2024-01-10 10:00:00'::TIMESTAMP;
-- Returns interval: 5 days 04:30:00

-- Convert interval to numeric seconds/hours
SELECT EXTRACT(EPOCH FROM INTERVAL '2 hours 30 minutes'); -- 9000 (seconds)
SELECT EXTRACT(EPOCH FROM (timestamp2 - timestamp1)) -- duration in seconds
FROM events;
```

Extraction and Truncation

date_part / EXTRACT

```
SELECT EXTRACT(year FROM '2024-07-15 14:30:00'::TIMESTAMP); -- 2024
SELECT EXTRACT(month FROM '2024-07-15 14:30:00'::TIMESTAMP); -- 7
SELECT EXTRACT(day FROM '2024-07-15 14:30:00'::TIMESTAMP); -- 15
SELECT EXTRACT(hour FROM '2024-07-15 14:30:00'::TIMESTAMP); -- 14
SELECT EXTRACT(minute FROM '2024-07-15 14:30:00'::TIMESTAMP); -- 30
SELECT EXTRACT(second FROM '2024-07-15 14:30:00'::TIMESTAMP); -- 0
SELECT EXTRACT(dow FROM '2024-07-15'::DATE); -- 1 (Monday;
0=Sunday)
SELECT EXTRACT(doy FROM '2024-07-15'::DATE); -- 197 (day of
year)
SELECT EXTRACT(week FROM '2024-07-15'::DATE); -- 29 (ISO week)
SELECT EXTRACT(quarter FROM '2024-07-15'::DATE); -- 3
SELECT EXTRACT(epoch FROM now()); -- Unix timestamp
(seconds)

-- date_part is older syntax, same result
SELECT date_part('year', now());
```

date_trunc

```
SELECT date_trunc('year', '2024-07-15 14:30:45'::TIMESTAMP); -- 2024-01-01
00:00:00
SELECT date_trunc('month', '2024-07-15 14:30:45'::TIMESTAMP); -- 2024-07-01
00:00:00
SELECT date_trunc('week', '2024-07-15 14:30:45'::TIMESTAMP); -- 2024-07-15
(Monday)
SELECT date_trunc('day', '2024-07-15 14:30:45'::TIMESTAMP); -- 2024-07-15
00:00:00
SELECT date_trunc('hour', '2024-07-15 14:30:45'::TIMESTAMP); -- 2024-07-15
14:00:00
SELECT date_trunc('minute', '2024-07-15 14:30:45'::TIMESTAMP); -- 2024-07-15
14:30:00

-- Useful for grouping by time period
SELECT
    date_trunc('month', created_at) AS month,
    count(*) AS signups
FROM users
GROUP BY 1
ORDER BY 1;
```

Special Values and Constants

```
-- Current date/time functions
SELECT CURRENT_DATE; -- current date (DATE type)
SELECT CURRENT_TIME; -- current time with timezone (TIMETZ)
```

```
SELECT CURRENT_TIMESTAMP;      -- current timestamp with timezone (TIMESTAMP TZ)
SELECT LOCALTIME;              -- current time without timezone
SELECT LOCALTIMESTAMP;         -- current timestamp without timezone
SELECT now();                  -- same as CURRENT_TIMESTAMP, SQL-friendly

-- These are stable within a transaction:
BEGIN;
SELECT now(); -- same value throughout this transaction
SELECT pg_sleep(1);
SELECT now(); -- still same value!
COMMIT;

-- For the actual wall clock time:
SELECT clock_timestamp(); -- changes with each call, even in a transaction
SELECT statement_timestamp(); -- stable within a statement, changes between
statements

-- Special date values
SELECT 'epoch'::TIMESTAMP;      -- 1970-01-01 00:00:00
SELECT 'infinity'::TIMESTAMP; -- positive infinity
SELECT '-infinity'::TIMESTAMP; -- negative infinity
SELECT 'now'::TIMESTAMP;       -- resolves at parse time (avoid in views/functions!)
SELECT 'today'::DATE;
SELECT 'tomorrow'::DATE;
SELECT 'yesterday'::DATE;
```

Storage Reference

Type	Storage	Min	Max	Resolution
date	4 bytes	4713 BC	5874897 AD	1 day
time	8 bytes	00:00:00	24:00:00	1 microsecond
timetz	12 bytes	00:00:00+1459	24:00:00-1459	1 microsecond
timestamp	8 bytes	4713 BC	294276 AD	1 microsecond
timestamp tz	8 bytes	4713 BC	294276 AD	1 microsecond
interval	16 bytes	-178M years	+178M years	1 microsecond

SQL Examples

Events system with comprehensive date/time usage

```
-- Events table with timezone-aware timestamps
CREATE TABLE events (
  event_id      BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title         TEXT        NOT NULL,
```

```

starts_at      TIMESTAMPTZ      NOT NULL,
ends_at        TIMESTAMPTZ      NOT NULL,
-- Store the original timezone for display purposes
display_timezone TEXT           NOT NULL DEFAULT 'UTC',
created_at     TIMESTAMPTZ      NOT NULL DEFAULT now(),
CONSTRAINT valid_duration CHECK (ends_at > starts_at),
CONSTRAINT reasonable_duration CHECK (ends_at - starts_at <= INTERVAL '30 days')
);

-- Recurring schedule: store just dates for things that don't need time
CREATE TABLE recurring_holidays (
  holiday_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name       TEXT      NOT NULL,
  holiday_date DATE     NOT NULL,
  -- Recurring: month and day only
  recurring_month SMALLINT CHECK (recurring_month BETWEEN 1 AND 12),
  recurring_day   SMALLINT CHECK (recurring_day BETWEEN 1 AND 31)
);

-- Time-series data
CREATE TABLE metrics (
  metric_id BIGINT      GENERATED ALWAYS AS IDENTITY,
  sensor_id  INTEGER     NOT NULL,
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  value      DOUBLE PRECISION NOT NULL,
  PRIMARY KEY (sensor_id, recorded_at)
) PARTITION BY RANGE (recorded_at);

-- Monthly partitions
CREATE TABLE metrics_2024_01
  PARTITION OF metrics
  FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- Useful reporting queries
-- Events in the next 7 days
SELECT *
FROM events
WHERE starts_at BETWEEN now() AND now() + INTERVAL '7 days';

-- Group by month
SELECT
  to_char(date_trunc('month', starts_at), 'YYYY-MM') AS month,
  count(*) AS event_count
FROM events
GROUP BY 1
ORDER BY 1;

-- Events happening right now
SELECT *
FROM events
WHERE now() BETWEEN starts_at AND ends_at;

```



```
-- Convert to a specific timezone for display
SELECT
    title,
    starts_at AT TIME ZONE display_timezone AS local_start
FROM events;
```

Common Mistakes

1. Using `timestamp` (no TZ) instead of `timestamptz`

```
-- BAD: loses timezone context, causes bugs on server moves or DST
created_at TIMESTAMP NOT NULL DEFAULT now()

-- GOOD: always timezone-aware
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

2. Using `'now'::TIMESTAMP` as a default

```
-- BAD: 'now' is resolved at parse time, not at insert time
-- All rows in a table get the same timestamp when the definition is parsed
created_at TIMESTAMP DEFAULT 'now'

-- GOOD: function is called at insert time
created_at TIMESTAMPTZ DEFAULT now()
-- Or:
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
```

3. Performing arithmetic on dates and expecting date result

```
SELECT '2024-01-15'::DATE + INTERVAL '1 month';
-- Returns: 2024-02-15 00:00:00 (TIMESTAMP, not DATE!)
-- Fix:
SELECT ('2024-01-15'::DATE + INTERVAL '1 month')::DATE;
```

4. Ignoring DST when scheduling — "Every day at 9 AM" should be stored as a recurring rule, not a fixed timestamp interval. — `now() + INTERVAL '1 day'` is 23 or 25 hours during DST transitions.

5. Comparing timestamps with different types

```
-- This works but has an implicit cast
WHERE created_at = '2024-01-15'
-- Better: be explicit about the type
WHERE created_at >= '2024-01-15'::DATE
AND created_at < '2024-01-16'::DATE
```

6. Using `EXTRACT(epoch FROM ...)` for duration in seconds — but forgetting about DST — EPOCH extraction from an interval gives consistent seconds, but for `timestamptz` differences it correctly accounts for DST.

Best Practices

1. **Always use `TIMESTAMPZ`** for timestamps in application tables.
 2. **Use `DATE`** for calendar dates where time is irrelevant (birthdays, holidays, delivery dates).
 3. **Store the application timezone** in a separate column when you need to display times in the original timezone.
 4. **Set `timezone = 'UTC'`** in `postgresql.conf` for servers to avoid server-timezone-dependent behavior.
 5. **Use `date_trunc()`** for grouping and period calculations.
 6. **Use `INTERVAL`** for relative time calculations — avoid computing seconds manually.
 7. **Index timestamp columns** that are used in WHERE clauses with BRIN or B-tree indexes.
 8. **Partition time-series tables** by time range for efficient pruning.
-

Performance Considerations

Index strategies for time columns

```
-- B-tree (default): best for equality, range queries, ORDER BY
CREATE INDEX idx_events_starts ON events (starts_at);

-- BRIN index: very compact, good for naturally ordered timestamp data (time-series)
CREATE INDEX idx_metrics_recorded_brin ON metrics USING BRIN (recorded_at);
-- BRIN is ~100x smaller than B-tree for sequential data

-- Partial index: common pattern for "recent" data
CREATE INDEX idx_events_upcoming ON events (starts_at)
WHERE starts_at >= now(); -- Note: can't use function in partial index predicate
this way
-- Better:
CREATE INDEX idx_events_active ON events (starts_at)
WHERE ends_at > '2024-01-01'; -- static cutoff, updated periodically

-- Composite index for common query pattern
CREATE INDEX idx_events_user_time ON events (user_id, starts_at DESC);
```

Range query performance

```
-- GOOD: range query uses index
SELECT * FROM events
WHERE starts_at >= '2024-01-01' AND starts_at < '2024-02-01';

-- GOOD: equivalent using date_trunc
SELECT * FROM events
WHERE date_trunc('month', starts_at) = '2024-01-01';
-- But this may not use the index! Use range form above for index benefit.

-- GOOD: BETWEEN (inclusive on both ends)
```

```
SELECT * FROM events
WHERE starts_at BETWEEN '2024-01-01' AND '2024-01-31 23:59:59.999999';
```

Interview Questions & Answers

Q1: What is the difference between `timestamp` and `timestamptz` in PostgreSQL?

A: `timestamp` stores a date and time with no timezone information. The value is stored and retrieved exactly as given. `timestamptz` converts the input value to UTC at storage time, then converts back to the session timezone at display time. Both use 8 bytes of storage. `timestamptz` is almost always the correct choice for application code.

Q2: If `timestamptz` uses the same 8 bytes as `timestamp`, where is the timezone stored?

A: It isn't stored with the value — the timezone is not part of the stored data. `timestamptz` stores a UTC value. The display timezone is the session's current timezone setting. This means two sessions with different timezone settings will display the same `timestamptz` value differently, but they are comparing the same underlying UTC instant.

Q3: Why is `timetz` (time with time zone) considered useless?

A: A time of day with a timezone offset but no date cannot be used for correct DST-aware calculations. For example, "14:30:00+05:30" without a date cannot tell you whether that's during DST or standard time. The type exists for SQL standard compliance. In practice, use `timestamptz` for timezone-aware temporal data.

Q4: Why are months, days, and microseconds stored separately in an interval?

A: Because these units are not constant-length. A month can be 28, 29, 30, or 31 days. A day can be 23 or 25 hours during DST transitions. Storing them separately allows correct arithmetic: `'2024-01-31'::DATE + INTERVAL '1 month'` correctly gives `2024-02-29` (or `2024-02-28` if not a leap year).

Q5: What is the result of `now()` inside a transaction?

A: `now()` returns the transaction start time and remains constant throughout the transaction.
`clock_timestamp()` returns the actual wall clock time and changes with each call.
`statement_timestamp()` changes between statements but stays constant within a statement.

Q6: How do you find all records created "today" efficiently?

A: Use a range query on `CURRENT_DATE`:

```
WHERE created_at >= CURRENT_DATE AND created_at < CURRENT_DATE + 1
```

Avoid `date_trunc('day', created_at) = CURRENT_DATE` because wrapping the column in a function prevents index usage.

Q7: What is BRIN index and when is it best for timestamps?

A: BRIN (Block Range Index) stores min/max values for each range of disk blocks. It works best when data is physically ordered on disk — as time-series data typically is (new rows appended chronologically). BRIN is much smaller than B-tree (hundreds of KB vs. hundreds of MB) but only efficient when the physical data order correlates with the query range.

Q8: How would you calculate the age of an account in years and months?

A: `SELECT age(CURRENT_DATE, created_date::DATE)` returns an interval in years/months/days format. To extract just years: `EXTRACT(year FROM age(CURRENT_DATE, created_date))`.

Q9: What timezone should PostgreSQL servers use?

A: UTC. Setting `timezone = 'UTC'` in `postgresql.conf` ensures consistent behavior regardless of the server's OS timezone, avoids DST surprises, and makes it easy to reason about stored data. The application layer handles conversion to local timezones for display.

Q10: How do you generate a series of dates?

A: Use `generate_series()`:

```
SELECT generate_series(
    '2024-01-01'::DATE,
    '2024-12-31'::DATE,
    '1 day'::INTERVAL
)::DATE AS calendar_date;
```

Exercises with Solutions

Exercise 1

Create a `bookings` table for a hotel reservation system. Ensure check-out is after check-in, enforce that bookings cannot be more than 30 days, and add a generated column for the number of nights.

Solution:

```
CREATE TABLE bookings (
    booking_id      BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    guest_id        INTEGER     NOT NULL,
    room_id         INTEGER     NOT NULL,
    check_in        DATE        NOT NULL,
    check_out       DATE        NOT NULL,
    nights          INTEGER     GENERATED ALWAYS AS (check_out - check_in) STORED,
    booked_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    -- Constraints
    CONSTRAINT valid_dates CHECK (check_out > check_in),
    CONSTRAINT max_stay   CHECK (check_out - check_in <= 30),
    CONSTRAINT no_past_checkin CHECK (check_in >= CURRENT_DATE)
);
```

Exercise 2

Write a query that buckets orders into time periods: last hour, today, this week, this month, older.

Solution:

```
SELECT
    order_id,
    created_at,
    CASE
```

```

        WHEN created_at >= now() - INTERVAL '1 hour' THEN 'Last Hour'
        WHEN created_at >= CURRENT_DATE THEN 'Today'
        WHEN created_at >= date_trunc('week', now()) THEN 'This Week'
        WHEN created_at >= date_trunc('month', now()) THEN 'This Month'
        ELSE 'Older'
    END AS time_bucket
FROM orders
ORDER BY created_at DESC;

```

Exercise 3

Write a query to find "business days" between two dates (Mon–Fri, excluding weekends).

Solution:

```

WITH date_range AS (
    SELECT generate_series(
        '2024-01-01'::DATE,
        '2024-01-31'::DATE,
        '1 day'::INTERVAL
    )::DATE AS d
)
SELECT
    count(*) AS business_days
FROM date_range
WHERE EXTRACT(dow FROM d) NOT IN (0, 6); -- 0 = Sunday, 6 = Saturday

```

Cross-References

- [01_numeric_types.md](#) — EXTRACT returns numeric values
- [09_sequences_identity.md](#) — auto-generated timestamps
- [08_constraints.md](#) — CHECK constraints on date ranges
- [../07_Indexes/](#) — BRIN indexes for time-series data
- [../16_PostgreSQL_For_Data_Engineering/](#) — time-series partitioning patterns